ERHVERVSAKADEMI KØBENHAVN

Week 3 — Functions

Text • Numeric • Date/Time • System

Today's Plan

Warm-up: recap
Text functions: normalize names, extract parts, validate strings
Numeric functions: rounding, ABS, simple rules (CASE)
Date/time functions: age/days since, add/subtract intervals
System functions: quick checks (database/user/version)
Exit ticket: one query combining multiple functions

Setup: Run the Handout Script

- **DBeaver:**
- • Open a SQL Editor in DBeaver (your MariaDB connection)
- • Run: lesson3_functions_handout.sql
- • Set active database/schema: school
- • Tip: run SELECT DATABASE(); to confirm

- **Quick checks:**
- SELECT COUNT(*) AS students FROM school.students;
- SELECT COUNT(*) AS enrollments FROM school.enrollments;

- **Tables you will use today:**
- students (names, email, birthdate, created_at)
- courses (course_code, title, credits)
- enrollments (enrolled_on, final_grade, notes)

Why Functions Matter

- **Functions transform a value into another value (often 'on the fly' in SELECT)**

- They let you:
- • Standardize messy inputs: casing, spaces, formats
- • Compute helpful fields: 'age', 'days since', rounded grades
- • Reduce work in the application layer

- **Mental model:**
- Input value → FUNCTION(value) → output value

- Example: SELECT UPPER('alice') AS result; -- result = 'ALICE'

- Scalar functions return one value per row

EK ERHVERVSAKADEMI KØBENHAVN

Function Patterns — SELECT + Aliases + Nesting

- **Syntax:**
- • Use functions in SELECT to create computed output columns
- • Always alias computed columns (readability + later reuse)
- • Functions can be nested: f(g(x)). Start from the inside
- • Be aware of data types: strings vs numbers vs dates

- **Example:**
- SELECT student_id,
- UPPER(TRIM(last_name)) AS last_upper
- FROM students;
- -- TRIM runs first, then UPPER

- **Rule of thumb:**
- Write the inner function first, test it, then wrap it

EK ERHVERVSAKADEMI KØBENHAVN

NULL Handling: Avoid 'Surprise NULL' Outputs

- **NULL means 'unknown/missing' (not the same as empty string)**

- • Many functions propagate NULL: result becomes NULL if input is NULL
- • In MariaDB, CONCAT() returns NULL if any argument is NULL
- • Use COALESCE(x, default) or IFNULL(x, default) for safe display
- • Tip: CONCAT_WS(sep, ...) skips NULL arguments

- **Example — Surprise NULL:**
- SELECT CONCAT('Email: ', email) AS msg FROM students;

- **Safer display:**
- SELECT CONCAT('Email: ', COALESCE(email, '(none)')) AS msg
- FROM students;

Text Functions — Strings

- **Common string-cleaning tasks:**
- • Case conversion: UPPER(), LOWER()
- • Trimming spaces: TRIM()
- • Length checks: LENGTH()
- • Substrings: LEFT(), RIGHT(), SUBSTRING()
- • Search/replace: LOCATE(), REPLACE()
- • Combine fields: CONCAT(), CONCAT_WS()

- **Example:**
- SELECT first_name,
- UPPER(TRIM(last_name)) AS last_upper,
- LENGTH(TRIM(first_name)) AS fn_len
- FROM students;

Example: Normalize Names for Reporting

- **Goal:**
- • last_name in ALL CAPS
- • first_name in 'Proper case' (First letter uppercase, rest lowercase)
- • email safely displayed even if NULL

- SELECT student_id,
- CONCAT(
- UPPER(LEFT(TRIM(first_name), 1)),
- LOWER(SUBSTRING(TRIM(first_name), 2))
-) AS first_proper,
- UPPER(TRIM(last_name)) AS last_upper,
- COALESCE(email, '(no email)') AS safe_email
- FROM students ORDER BY student_id;

Exercise A

Create a SELECT for students with columns:
• first_name_proper (TRIM + UPPER/LOWER + LEFT/SUBSTRING)
• last_name_upper (TRIM + UPPER)
• safe_email (COALESCE to show '(no email)')

Add an alias for every computed column
Stretch: use CONCAT_WS to build safe_full_name

Handout: lesson3_functions_handout.sql

Text: Extracting Parts of a String

- • Use LEFT/RIGHT when you know fixed lengths
- • Use SUBSTRING when you know a starting position
- • Use LOCATE (or INSTR) to find a separator like '-'
- • For 'prefix-suffix' codes, split around the dash

- SELECT course_code,
- LEFT(course_code, 3) AS first_3,
- LOCATE('-', course_code) AS dash_pos,
- SUBSTRING(course_code, 1, LOCATE('-', course_code) - 1) AS prefix,
- SUBSTRING(course_code, LOCATE('-', course_code) + 1) AS suffix
- FROM courses;

- **LOCATE returns 0 if not found → handle with CASE when validating**

EK ERHVERVSAKADEMI KØBENHAVN

Text Validation: Flag Suspect Strings

- • Validation is often a computed 'flag' column
- • Use LENGTH(TRIM(x)) to avoid spaces faking a valid value
- • CASE WHEN … THEN … ELSE … END creates readable rules

- SELECT student_id, first_name,
- LENGTH(TRIM(COALESCE(first_name, ''))) AS fn_len,
- CASE
- WHEN first_name IS NULL THEN 'MISSING'
- WHEN LENGTH(TRIM(first_name)) < 2 THEN 'TOO_SHORT'
- WHEN LENGTH(TRIM(first_name)) > 25 THEN 'TOO_LONG'
- ELSE 'OK'
- END AS first_name_flag
- FROM students;

Exercise B

For courses, create a SELECT with:
- course_code
- prefix (before the dash)
- suffix (after the dash)
- code_flag (OK / WRONG_LENGTH / MISSING_DASH)

Then make a 'safe display' column:
COALESCE(notes, '(no notes)') in enrollments

Numeric Functions

- **Common numeric-cleanup tasks:**
- • ROUND(x, decimals) to standardize numeric output
- • CEIL(x) / FLOOR(x) for rounding directions
- • ABS(x) for absolute difference ('distance')
- • MOD(a, b) for remainders

- SELECT enrollment_id, final_grade,
- ROUND(final_grade, 1) AS grade_round_1,
- ABS(final_grade - 7.00) AS abs_diff_from_pass
- FROM enrollments;

- **Think like a data analyst:**
- 'Do we want a number for calculation, or display?'
- Round only for display or explicitly for rules

EK ERHVERVSAKADEMI KØBENHAVN

Numeric Example: 'Distance from Pass'

- **Rule: pass is 7.0**
- • If grade is NULL → distance is NULL
- • If grade >= 7 → distance is 0
- • Else → (7 - grade), rounded to 2 decimals

- SELECT enrollment_id, final_grade,
- CASE
- WHEN final_grade IS NULL THEN NULL
- WHEN final_grade >= 7.0 THEN 0
- ELSE ROUND(7.0 - final_grade, 2)
- END AS distance_to_pass
- FROM enrollments ORDER BY enrollment_id;

- **CASE is your 'business rule' tool**

Exercise C

Write a SELECT over enrollments with:
• final_grade rounded to 1 decimal
• abs_diff_from_7 (ABS(final_grade - 7))
• distance_to_pass (the CASE rule from the demo)

Stretch: add grade_band
(e.g., 0-4, 4-7, 7-10, 10-12) using CASE

Handout: lesson3_functions_handout.sql

Date/Time Functions

- **Common date/time operations:**
- • Get current time: NOW(), CURRENT_TIMESTAMP, CURDATE()
- • Extract parts: YEAR(date), MONTH(date), DAY(date)
- • Differences: DATEDIFF(date1, date2), TIMESTAMPDIFF(unit, a, b)
- • Add/subtract: DATE_ADD(date, INTERVAL n unit), DATE_SUB(...)

- SELECT CURDATE() AS today,
- NOW() AS now,
- YEAR(CURDATE()) AS yyyy,
- MONTH(CURDATE()) AS mm;

- **Two different 'differences':**
- • DATEDIFF gives number of days (date-to-date)
- • TIMESTAMPDIFF lets you choose units (e.g., YEAR for age)

EK ERHVERVSAKADEMI KØBENHAVN

Date/Time Example: Age and 'Days Since Created'

- • **Age is not 'days/365' → use TIMESTAMPDIFF(YEAR, …)**
- • created_at is DATETIME → use DATE(created_at) if you want date-only
- • Always handle NULL birthdates

- SELECT student_id, created_at,
- DATEDIFF(CURDATE(), DATE(created_at)) AS days_since_created,
- birthdate,
- CASE
- WHEN birthdate IS NULL THEN NULL
- ELSE TIMESTAMPDIFF(YEAR, birthdate, CURDATE())
- END AS age_years
- FROM students;

- **Predict-first habit: Pick one student and estimate days_since_created before running**

EK ERHVERVSAKADEMI KØBENHAVN

Date/Time: Add/Sub Intervals + Bucket by Month

- • DATE_ADD/DATE_SUB are good for deadlines and follow-ups
- • Bucketing is often computed as YYYY-MM (grouping comes later)
- • We compute the bucket now, then later we GROUP BY it

- SELECT enrollment_id, enrolled_on,
- DATE_ADD(enrolled_on, INTERVAL 14 DAY) AS followup_plus_14,
- CONCAT(YEAR(enrolled_on), '-',
- LPAD(MONTH(enrolled_on), 2, '0')) AS enrolled_yyyymm
- FROM enrollments ORDER BY enrolled_on;

- **LPAD makes months 01..12 for consistent sorting**

- Design note: For heavy reporting, you might store derived fields or build a view

EK ERHVERVSAKADEMI KØBENHAVN

Exercise D

Create a SELECT over students with:
• days_since_created
• age_years (NULL-safe)

Then create a SELECT over enrollments with:
• enrolled_yyyymm (YYYY-MM)
• followup_plus_14_days

Stretch: show 'created_month' as YYYY-MM from created_at

System Functions — Environment & Session

- **Useful for troubleshooting and logging:**
- • DATABASE() — which schema am I using?
- • VERSION() — which MariaDB version?
- • USER() / CURRENT_USER() — who am I connected as?

- SELECT DATABASE() AS db,
- VERSION() AS version,
- USER() AS session_user,
- CURRENT_USER() AS current_user;

- **When to use these in practice:**
- • Bug report: include VERSION() output
- • Support scripts: assert DATABASE() before running destructive commands
- • Auditing: store USER() in a log table (later topic)

EK ERHVERVSAKADEMI KØBENHAVN

Combining Functions — Nested + NULL-Safe Output

- **Goal: one readable output field for a report/export**
- • Use CONCAT_WS to avoid NULL parts
- • Use COALESCE to provide defaults for missing values
- • Keep intermediate pieces as columns while debugging

- SELECT e.enrollment_id,
- CONCAT_WS(' — ',
- CONCAT_WS(' ',
- CONCAT(UPPER(LEFT(TRIM(s.first_name), 1)),
- LOWER(SUBSTRING(TRIM(s.first_name), 2))),
- UPPER(TRIM(s.last_name))),
- c.course_code,
- CONCAT('grade ', COALESCE(ROUND(e.final_grade, 1), 'N/A'))
-) AS display_line

Exit Ticket

Write ONE query that uses at least 3 different functions

Must include:
- One text function
- One numeric OR date/time function
- COALESCE

Explain (in one sentence) what problem your query solves

EK ERHVERVSAKADEMI KØBENHAVN

Summary + Next Steps

You should now be able to